

Video Game Programming: Camera System

L04

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

Game camera

- A camera or a set of cameras determine which parts of the game scene are visible to players
- Game cameras simulate real cameras filming a real-world scene
- Camera system determines:
 - Camera properties (e.g., projection, field of view,...)
 - Camera control (e.g., movement, rotation,...)

Geography and representation

- Camera system depends on different geography and representation design choices (in fig.)
- Perspective and dimensions influence camera properties
- Perspective, space type, scroll, and exploration influence camera control

2.5 isometric

Perspective	First person		Third person	
Dimensions	Two		Three	
Space type	Torus	Abstract	Free	
Off-screen Space	Dynamic	Static	None	
Scroll	Vertical	Horizontal	Free	None
Exploration	Forced	Free	None	

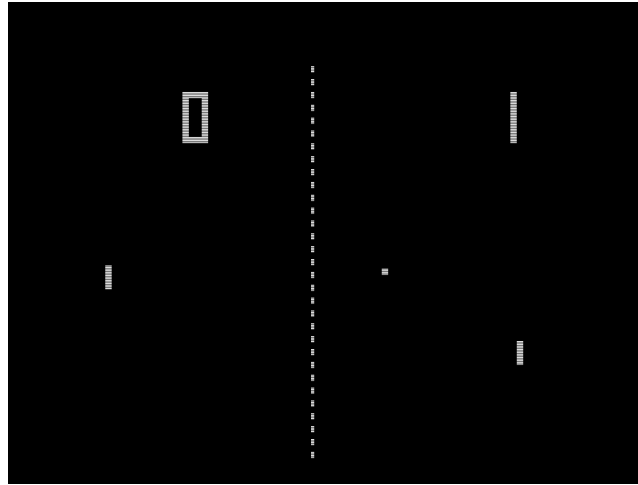
unconnected levels
zone-based multiscreen spaces
seamless multiscreen spaces

Perspective

- Except for text-based and abstract games, all games employ a first- or third-person perspective
- First-person perspective renders the scene from the point of view of player-controlled character, which can be visible (**avatar**) or not
- Third-person perspective shows the player-controlled character, objects, or settings in the scene
- Early games mostly used third-person perspective, while first-person became popular with 3D FPSs
- Genre and platform usually (not always) influence perspective choice
- Some games allow to switch from third- to first-person perspective

Dimensions

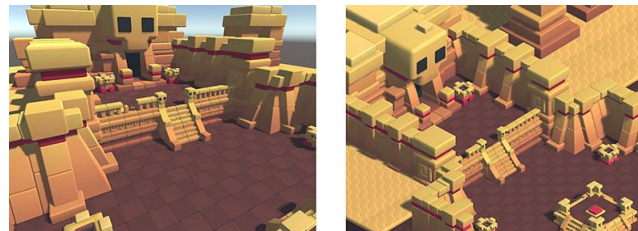
- 2D games usually simulate an orthographic (e.g., top-down or side-scroller) or an isometric projection (2.5D)
- 3D games can either use orthographic or perspective cameras



Top-down 2D camera in Pong



Isometric camera in SimCity



Same 3D scene using perspective or orthographic camera



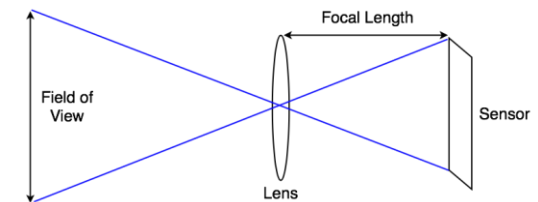
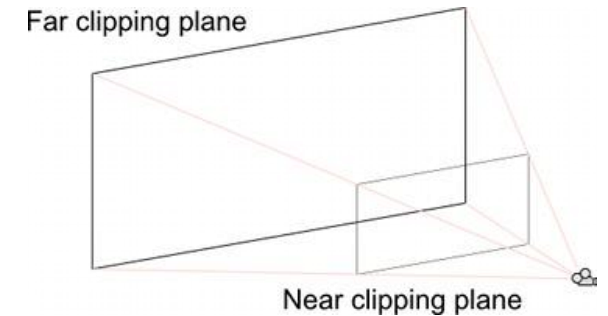
Perspective 3D camera in Ratchet&Clank

Space type, scroll, and exploration

- A torus space implies that camera should appear to the left when reached the extreme right (and viceversa)
- Scroll determines if the camera is fixed or if it can be moved up/down or left/right
- Exploration determines if the camera is fixed or if it is controlled by the player or automatically

Camera properties

- Projection can be either perspective or orthographic
- Field of view (FOV) of perspective cameras is the view angle in degrees
- Size determines visible area for orthographic cameras
- Near and far clipping planes determines which close and far objects are viewed or not
- FOV and clipping planes determine frustrum (in fig.)
- Physical camera properties can be used in some game engines (including Unity)



Camera control

- Cameras have a position and a rotation (aim direction)
- Considering control, there are 3 types of cameras:
 - **Fixed** cameras do not move, or their movement is predefined, e.g., when player changes area or in pre-rendered cutscenes
 - **Tracking** cameras automatically move, e.g., by following player-controlled character or by applying complex cinematic rules
 - **Interactive** cameras are controlled by the player, e.g., by moving the avatar in a first-person game or by scrolling in a third-person game

Automatic camera control

- Unity Cinemachine package
- Ranon and Urli, "Improving the Efficiency of Viewpoint Composition", IEEE Trans. Visualization and Computer Graphics, vol. 20, no. 5, pp. 795-807, 2014
- Application example in fig.:
Ranon, Chittaro, Buttussi,
"Automatic camera control
meets emergency simulations:
An application to aviation
safety", Computers & Graphics,
vol. 48, pp. 23-34, 2015



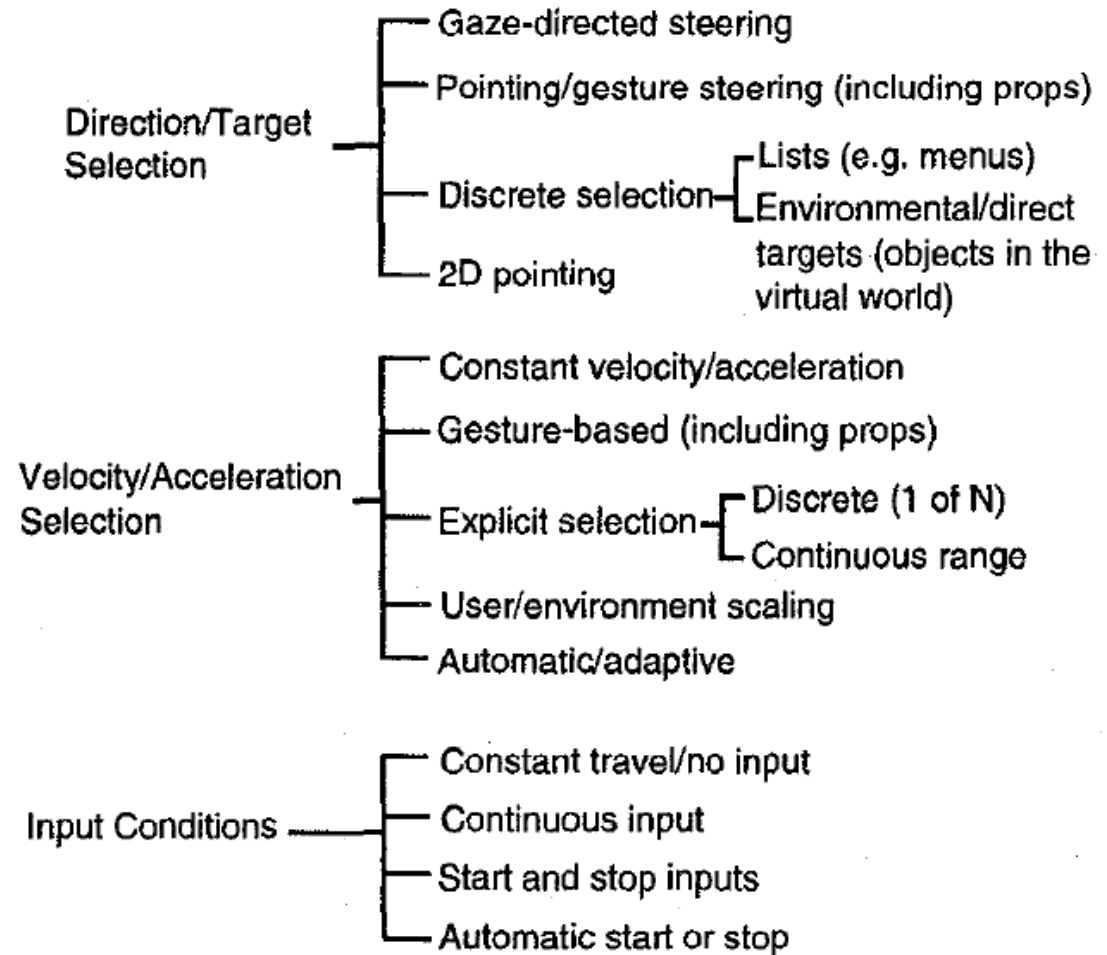
<https://youtu.be/DJq87oasil8>

Input devices

- Interactive cameras are directly controller by the players
- Tracking cameras can be updated based on players' actions
- In both cases, players' interaction with input devices affects camera control
- Different platforms support different input devices:
 - PC: mouse, keyboard, joystick, gamepads,...
 - Console: gamepads, cameras, wheels, tracked controllers, balance boards,...
 - Mobile: touchscreen, accelerometer, gyroscope, cameras,...
 - VR: 6-DOF head tracker, gamepads, tracked controllers, cameras, omni-directional treadmills,...

Locomotion techniques for first-person cameras

- To move first-person cameras in virtual environments programmers must implement a locomotion technique
- A taxonomy was provided in Bowman, Koller, Hodges, “Travel in immersive virtual environments: an evaluation of viewpoint motion control techniques”, Procs. IEEE 1997 Ann. Intl. Symp. Virtual Reality, pp. 45-52, 1997 (in fig.)

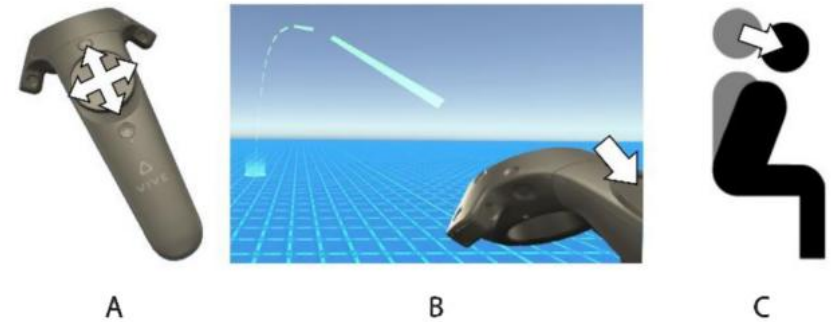


Most-used locomotion techniques

- In immersive VR, the most used locomotion techniques are:
 - Steering (using joystick, pointing, gaze, walk-in-place,...)
 - Teleport (pointing, choosing among predefined alternatives,...)
 - Real or redirected walking (using head trackers and redirection algorithms)
- Steering is typically used on PC and consoles (e.g., using gamepads) as well as on mobile (e.g., using virtual joysticks on the touchscreen)

Implications of locomotion technique

- Implemented locomotion technique may affect players' performance, sickness, presence, usability, comfort,...
- A study comparing locomotion techniques in VR as well as previous studies from the literature are described in Buttussi, Chittaro, "Locomotion in Place in Virtual Reality: A Comparative Evaluation of Joystick, Teleport, and Leaning", IEEE Trans. Vis. Comput. Graph., vol. 27, pp. 125-136, 2021



Control of third-person cameras

- With tracking cameras, players typically use a locomotion technique (typically steering) to move the main character or object, and the camera automatically follows (in fig.)
- In games where users can interact with several characters, objects, or settings, third-person camera can follow selected characters or be interactively controlled by the player (in fig.)



Tracking camera in
Crash Bandicoot



Interactive camera in
AeroSims (Air Safety World)

Bibliografy

- Egenfeldt-Nielsen, Smith, Tosca, Understanding Video Games: The Essential Introduction, 5th ed., Routledge, 2024
- Gomila Salas Juan Gabriel, Borromeo Nicolas Alejandro, Hands-On Unity Game Development, 4th ed., Packt Publishing, 2024
- Murray, C# Game Programming Cookbook for Unity 3D, 2nd ed., CRC Press, 2021

